

# Wall Damage Detection and Cost Estimation System Documentation

Wall Damage Project Team

July 14, 2025

## Contents

|          |                                                 |          |
|----------|-------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                             | <b>2</b> |
| 1.1      | Purpose and Scope . . . . .                     | 2        |
| 1.2      | Rationale for Approach . . . . .                | 2        |
| <b>2</b> | <b>Technologies Used</b>                        | <b>2</b> |
| <b>3</b> | <b>Libraries and Tools Used</b>                 | <b>3</b> |
| <b>4</b> | <b>System Architecture</b>                      | <b>5</b> |
| <b>5</b> | <b>Implementation Steps</b>                     | <b>6</b> |
| 5.1      | Step 1: Defining Project Requirements . . . . . | 6        |
| 5.2      | Step 2: Data Collection . . . . .               | 6        |
| 5.3      | Step 3: Selecting Damage Types . . . . .        | 6        |
| 5.4      | Step 4: Data Annotation . . . . .               | 7        |
| 5.5      | Step 5: Model Selection . . . . .               | 7        |
| 5.6      | Step 6: Model Training . . . . .                | 7        |
| 5.7      | Step 7: Model Evaluation . . . . .              | 7        |
| 5.8      | Step 8: Cost Estimation Module . . . . .        | 7        |
| 5.9      | Step 9: Explanation Generation . . . . .        | 8        |
| 5.10     | Step 10: Backend Development . . . . .          | 8        |
| 5.11     | Step 11: Environment Setup . . . . .            | 8        |
| 5.12     | Step 12: Testing and Validation . . . . .       | 8        |
| 5.13     | Step 13: Deployment Preparation . . . . .       | 8        |
| 5.14     | Step 14: Fine-Tuning and Deployment . . . . .   | 8        |
| <b>6</b> | <b>Challenges and Solutions</b>                 | <b>9</b> |
| <b>7</b> | <b>Model Outputs</b>                            | <b>9</b> |
| <b>8</b> | <b>Tools and Technologies Summary</b>           | <b>9</b> |
| <b>9</b> | <b>Conclusion</b>                               | <b>9</b> |

# 1 Introduction

The Wall Damage Detection and Cost Estimation System is an AI-powered solution designed to detect, classify, and estimate repair costs for wall damages in Sri Lankan buildings, while providing actionable repair instructions. The system targets four damage types: crack damages, flaking paint, missing pieces, and water damage. It aims to assist homeowners, contractors, and property managers by automating assessments, reducing reliance on manual inspections, which are time-consuming, costly, and prone to human error. The project integrates computer vision, web technologies, and natural language processing to deliver accurate, scalable, and user-friendly results, targeting a mean Average Precision (mAP) above 0.8 by July 30, 2025.

## 1.1 Purpose and Scope

The project was developed to address the need for efficient, cost-effective wall damage assessment in Sri Lanka, where localized repair costs (e.g., 20,000 LKR/m<sup>2</sup> for cracks) are critical for practical application. By combining YOLOv8 nano for object detection, ResNet-50 for classification, a cost estimation module, and the Google Gemini API for repair instructions, the system ensures precision and accessibility. The scope includes image-based damage detection, classification, area estimation, cost calculation in LKR, and structured repair guidance.

## 1.2 Rationale for Approach

An automated system combining machine learning and web technologies was chosen to ensure scalability, accuracy, and accessibility. Computer vision models (YOLOv8 and ResNet-50) provide precise damage identification, a NestJS backend enables web-based access, and the Gemini API generates user-friendly explanations, offloading complex NLP tasks to an external service. This approach balances technical accuracy with practical usability, leveraging mature technologies to meet both professional and non-technical user needs.

# 2 Technologies Used

The system employs a robust stack of technologies, selected for their maturity, performance, and community support.

- **Node.js with NestJS:**

- **Purpose:** Backend framework for handling HTTP requests, file uploads, and Python script integration.
- **Rationale:** NestJS's modular, TypeScript-based architecture supports scalable APIs with dependency injection and middleware, ideal for integrating with Python scripts and handling image uploads. Node.js ensures high performance for I/O-bound tasks.

- **Python:**

- **Purpose:** Core language for machine learning, image processing, and API interactions.
- **Rationale:** Python's extensive ecosystem (PyTorch, TensorFlow, OpenCV, etc.) is the standard for computer vision and NLP tasks, offering flexibility for model inference and external API calls.
- **Conda:**
  - **Purpose:** Manages Python environments and dependencies.
  - **Rationale:** Conda ensures reproducible environments, isolating dependencies for machine learning ('wall\_damage', Python3.9.18) and API interactions ('LLM\_env', Python3.10), preventing conflicts.
- **Google Gemini 1.5 Flash API:**
  - **Purpose:** Generates structured JSON explanations for damage types.
  - **Rationale:** The Gemini API provides cost-effective, high-quality NLP with structured outputs, eliminating the need for local NLP models and aligning with the system's JSON output requirements.
- **Windows OS:**
  - **Purpose:** Development and training platform.
  - **Rationale:** Windows supports the required tools (Conda, Node.js, Python) and is compatible with CPU-based training, making it accessible for development without specialized hardware.

### 3 Libraries and Tools Used

The following libraries and tools were selected for their reliability, performance, and suitability.

- **ultralytics (YOLOv8, 8.3.133):**
  - **Purpose:** Object detection for identifying damage regions.
  - **Rationale:** YOLOv8 nano is lightweight, fast, and accurate, ideal for real-time detection. Its pre-trained weights and fine-tuning capabilities ensure high performance for wall damage localization.
- **tensorflow/keras:**
  - **Purpose:** Implements ResNet-50 for image classification.
  - **Rationale:** TensorFlow supports ResNet-50's deep architecture, leveraging pre-trained ImageNet weights for robust classification of damage types.
- **opencv-python (cv2):**
  - **Purpose:** Image preprocessing and resizing.
  - **Rationale:** OpenCV provides efficient tools for resizing images (e.g., 224x224 for ResNet-50, 640x640 for YOLOv8) and processing model outputs.

- **numpy:**
  - **Purpose:** Numerical computations for area and cost calculations.
  - **Rationale:** NumPys optimized array operations handle YOLOv8 and ResNet-50 outputs efficiently, supporting calculations like bounding box areas and confidence scores.
- **pandas:**
  - **Purpose:** Metadata and result handling.
  - **Rationale:** Pandas organizes image metadata and evaluation results into CSVs, simplifying data management and analysis.
- **scikit-learn:**
  - **Purpose:** Class weight balancing for ResNet-50 training.
  - **Rationale:** Scikit-learn improves classification accuracy by addressing class imbalances in the dataset.
- **python-shell:**
  - **Purpose:** Executes Python scripts from the NestJS backend.
  - **Rationale:** Enables seamless integration between Node.js and Python, passing image analysis results via JSON.
- **@nestjs/platform-express, multer:**
  - **Purpose:** Handles file uploads in the backend.
  - **Rationale:** Multer ensures secure file handling with validation (e.g., image formats, 10MB limit), integrating seamlessly with NestJS.
- **google.generativeai:**
  - **Purpose:** Interfaces with the Gemini API.
  - **Rationale:** Simplifies API calls for structured JSON explanations, aligning with the systems output format.
- **LabelImg:**
  - **Purpose:** Annotates images with bounding boxes for YOLOv8.
  - **Rationale:** LabelImg provides a user-friendly interface for precise bounding box annotations, essential for YOLOv8 training.
- **Jupyter Notebooks:**
  - **Purpose:** Exploratory data analysis and prototyping.
  - **Rationale:** Jupyter enables interactive testing of preprocessing and model pipelines, speeding up development.
- **Git:**
  - **Purpose:** Version control for scripts and models.

- **Rationale:** Git ensures collaborative development and tracking of changes, planned for future team expansion.
- **REST API, React.js, MongoDB :**
  - **Purpose:** API serving (FastAPI), front-end interface (React.js), and data storage (MongoDB).
  - **Rationale:** ensures cross-platform compatibility, REST API supports scalable APIs, React.js enables user-friendly interfaces, and MongoDB handles inference results efficiently.
- **Albumentations, Weights & Biases:**
  - **Purpose:** Advanced augmentations and experiment tracking for fine-tuning.
  - **Rationale:** Albumentations enhances model robustness, while Weights & Biases tracks performance improvements for mAP >0.8.

## 4 System Architecture

The Wall Damage Detection and Cost Estimation System follows a modular client-server architecture with four layers: client interface, backend processing, machine learning pipeline, and explanation generation. Each layer handles specific tasks, ensuring scalability, maintainability, and separation of concerns.

- **Client Interface Layer:**
  - **Functionality:** Users upload images via a POST request to the ‘/damage/-analyze’ endpoint, supporting formats (jpg, jpeg, png, gif, bmp, webp) with a 10MB limit. The layer returns JSON responses with damage type, area, cost, and repair instructions.
  - **Rationale:** A RESTful API ensures compatibility with web or mobile frontends, providing accessibility for diverse users. The endpoint-based design is standard and scalable.
- **Backend Processing Layer (NestJS):**
  - **Functionality:** Hosted at ‘/d/programming/fullstack development/wall\_damage\_backend’, the NestJS shell ‘It parses JSON outputs and returns structured responses.’ **Rationale :** NestJS modular architecture bridges Node.js and Python, chosen for its simplicity and reliability.
- **Machine Learning Pipeline (Python):**
  - **Functionality:** Implemented in ‘damage\_analyzer.py’, this layer uses YOLOv8nano(‘models/best\_50(‘models/resnet50\_classifier\_final.h5’) for classification, and custom logic for area(normalized) Python ecosystem supports advanced computer vision models. YOLOv8s localization and ResNet-50 classification ensure accuracy, while separate scripts enhance modularity.
- **Explanation Generation Layer (Gemini API):**
  - **Functionality:** Implemented in ‘LLM\_explanations.py’, this layer calls the Google Gemini 1.5 Flash. The external API reduces local NLP requirements, leveraging Gemini’s high-quality, structured and friendly instructions.

The layered architecture separates client interaction, backend processing, machine learning, and explanation generation, ensuring each component is independent and maintainable. The RESTful API, Python-based ML pipeline, and external NLP API balance performance, scalability, and development efficiency. Conda environments isolate dependencies, preventing conflicts between machine learning and API tasks.

## 5 Implementation Steps

The system was developed through a structured process, from project inception to deployment preparation. Each step is detailed below, including what was done, how it was implemented, and why the approach was chosen.

### 5.1 Step 1: Defining Project Requirements

- **What:** Defined the goal to detect four wall damage types (crack damages, flaking paint, missing pieces, water damage), estimate repair costs in LKR, and provide repair instructions, targeting mAP >0.8 by July 30, 2025.
- **How:** Conducted stakeholder interviews with contractors and homeowners in Sri Lanka to identify common damages and user needs (e.g., accuracy, localized costs, usability). Specified four damage types and system outputs (JSON with damage type, area, cost, instructions).
- **Rationale:** Focusing on prevalent damage types ensures practical applicability. Clear requirements guided technology selection, balancing complexity and utility for both technical and non-technical users.

### 5.2 Step 2: Data Collection

- **What:** Collected a diverse set of images representing the four damage types from construction sites and filtered online sources.
- **How:** Images were sourced from site visits in Sri Lanka and curated online repositories, ensuring variety in lighting, wall textures, and damage severities. Stored in `./i/AI model traing/WallDamageProject/yolo_dataset/images`.
- **Rationale :** *Diverse images ensure model robustness across world scenarios. Local sourcing aligns with Sri Lankan building characteristics, enhancing relevance.*

### 5.3 Step 3: Selecting Damage Types

- **What:** Chose crack damages, flaking paint, missing pieces, and water damage as target classes.
- **How:** Analyzed construction reports and consulted with local contractors to identify frequent damages. Selected types with distinct visual features (e.g., cracks vs. water stains) and varying repair costs (20,000-30,000 LKR/m<sup>2</sup>).
- **Rationale:** These types cover common scenarios, improving practical value. Distinct features enhance model accuracy, and varied costs justify automated estimation.

## 5.4 Step 4: Data Annotation

- **What:** Annotated images with bounding boxes for YOLOv8 and class labels for ResNet-50.
- **How:** Used LabelImg to create bounding box annotations in YOLO format, stored in `‘/i/AI model training/WallDamageProject/yolo_dataset/labels’`. Generated metadata CSVs (`‘train_metadata.csv’` for YOLOv8 and `‘train_metadata.csv’` for ResNet-50). Images were resized (640x640 for YOLOv8, 224x224 for ResNet-50) with OpenCV. **Rationale :** LabelImg ensures precise annotations for detection, while CSVs support classification. Resizing standardizes input sizes for both models.

## 5.5 Step 5: Model Selection

- **What:** Selected YOLOv8 nano for object detection and ResNet-50 for classification.
- **How:** Evaluated models (e.g., YOLOv5, Faster R-CNN, VGG16) based on accuracy, speed, and resource needs. Chose YOLOv8 nano for its lightweight, real-time performance and ResNet-50 for its deep architecture and ImageNet pre-training.
- **Rationale:** YOLOv8s speed and accuracy suit detection, while ResNet-50s robustness enhances classification. The dual-model approach ensures comprehensive damage identification.

## 5.6 Step 6: Model Training

- **What:** Trained YOLOv8 nano and ResNet-50 models on the annotated dataset.
- **How:** Trained YOLOv8 using the `‘ultralytics’` library (version 8.3.133) with a `‘data.yaml’` configuration in the `‘wall_damage’` Conda environment (Python 3.9.18), outputting `‘best.pt’` in `‘/i/AI model training/WallDamageProject/yolo_train/best.pt’`. Trained ResNet-50 with TensorFlow/Keras using transfer learning and class weight balancing (via `Scikit-learn`), saving `‘resnet50_classifier_final.h5’`. Training logs were stored in `‘yolo_train.log’` and `‘resnet_train.log’`. Fine-tuning leverages pre-trained weights, reducing training time. Conda ensures dependency compatibility across environments.

## 5.7 Step 7: Model Evaluation

- **What:** Evaluated YOLOv8 and ResNet-50 on the validation set to assess performance.
- **How:** Ran YOLOv8 validation with `‘ultralytics’` (`‘evaluate_yolo.py’`), producing detection metrics (`mAP50-95`) and CSVs (`‘yolo_detections.csv’`). Evaluated ResNet-50 with TensorFlow (`‘evaluate_resnet.py’`), generating classification metrics. Validation ensures model accuracy and identifies areas for improvement (e.g., `mAP`). CSVs facilitate result tracking.

## 5.8 Step 8: Cost Estimation Module

- **What:** Developed a module to estimate repair costs based on damage type and area.
- **How:** Implemented in `‘damage_analyzer.py’`, using YOLOv8 bounding box areas (normalized to 1m) and pre-defined repair rates. Localized rates ensure relevance for Sri Lanka. Normalized area estimations simplify calculations, and the module logs results for transparency.

## 5.9 Step 9: Explanation Generation

- **What:** Integrated the Google Gemini 1.5 Flash API to generate structured repair instructions.
- **How:** Developed `LLM_explanations.py` to query the Gemini API for JSON outputs (causes, resolution, etc.). The Gemini API provides high-quality, structured explanations without local NLP models, reducing complexity.

## 5.10 Step 10: Backend Development

- **What:** Built a NestJS backend for image uploads and analysis.
- **How:** Configured `app.module.ts` with Multer for file uploads (10MB limit, image format validation) at `/d/programming/fullstack development/wall_damage_backend`. Implemented `app.controller.ts` and `app.service.ts`. Stored images in `./uploads`. **Rationale :** NestJS ensures scalable, modular APIs. Multer secures uploads. `dotenv` simplifies Python integration, enabling seamless analysis.

## 5.11 Step 11: Environment Setup

- **What:** Set up two Conda environments for dependency management.
- **How:** Created `wall_damage` (Python 3.9.18) for YOLOv8, TensorFlow, OpenCV, NumPy, Pandas, and Keras, and `LLM_env` (Python 3.10) for `google.generativeai`. Configured `python-shell` to use the `wall_damage` environment at `/Anaconda3/envs/wall_damage/python.exe`. **Rationale :** Separate environments prevent dependency conflicts.

## 5.12 Step 12: Testing and Validation

- **What:** Tested the system end-to-end to ensure functionality and accuracy.
- **How:** Tested `damage_analyzer.py` standalone to verify JSON outputs, ran API tests with `curl` to the `/damage` endpoint. Created 50 evaluation scripts (`evaluate_yolo.py`, `evaluate_resnet.py`). Ensured Gemini API responses were structured. Comprehensive testing isolates issues and ensures reliability. Standalone and API tests verify integration and facing functionality.

## 5.13 Step 13: Deployment Preparation

- **What:** Prepared the system for deployment with organized files and documentation.
- **How:** Stored models in `/d/programming/fullstack development/wall_damage_backend/models/`, scripts in `scripts`. Organized structure and documentation ensure easy deployment and maintenance. Clear instructions support deployment.

## 5.14 Step 14: Fine-Tuning and Deployment

- **What:** to fine-tune models and deploy the system for real-time inference.
- **How:** Use Albumentations for advanced augmentations and Weights & Biases for experiment tracking to achieve mAP >0.8. Export YOLOv8 to (`best.pt`) for cross-platform compatibility. Deploy with REST API for API serving, React.js for the front-end, MongoDB for storing results (`deploy_detections.csv`, `deploy_classifications.csv`). **Rationale :** Fine-tuning improves accuracy, ensures portability, and REST API / React.js / MongoDB provide a scalable, friendly deployment stack. Monitoring tools ensure production reliability.

## 6 Challenges and Solutions

- **Challenge:** YOLOv8s stdout logs caused JSON parsing errors in the backend.
- **Solution:** Set ‘verbose=False’ in YOLOv8 and redirected logging to ‘*damage\_analyzer.log*’. This ensured consistent logging.
- **Challenge:** ResNet-50s low confidence scores in some cases.
  - **Solution:** Prioritized YOLOv8s output when its confidence is high ( $\geq 0.5$ ) or ResNet-50s is low ( $< 0.5$ ). This leverages YOLOv8s detection strength, chosen to maintain accuracy without retraining.
- **Challenge:** Integrating Python scripts with Node.js.
  - **Solution:** Used ‘python-shell’ to execute scripts and parse JSON outputs. This was chosen for its simplicity and compatibility with NestJS.

## 7 Model Outputs

- **YOLOv8 Outputs:**
  - \* **CSVs:** ‘*yolo\_detections.csv*’(*imagepaths*, *damagetypes*, *confidences*, *boundingboxareas*)  
*Annotated images with boundingboxes* (‘*runs/detect/val\_final/\*.png*’, ‘*runs/detect/deploy/\*.png*’)
  - \* **Logs:** Metrics (mAP50, mAP50-95) in ‘*yolo\_train.log*’.
- **ResNet-50 Outputs:**
  - \* **CSVs:** ‘*resnet\_classifications.csv*’(*imagepaths*, *damagetypes*, *confidences*) *for validation*  
*Accuracy and loss in* ‘*resnet\_train.log*’.
  - \* **History:** Training metrics in ‘*resnet50\_training\_historyV3.csv*’.
- **Cost Estimation Outputs:**
  - \* **CSV:** ‘*repair\_cost\_estimate.csv*’(*imagepaths*, *totalcostinLKR*, *damagetypes*, *areas*, *sources*).
- **Explanation Outputs:**
  - \* **JSON:** Structured responses from Gemini API (causes, resolution, steps, materials) integrated into API responses.
- **PDF Report:** Generated from this LaTeX document, summarizing the process, architecture, and outputs.

## 8 Tools and Technologies Summary

## 9 Conclusion

The Wall Damage Detection and Cost Estimation System integrates YOLOv8, ResNet-50, a NestJS backend, and the Gemini API to deliver automated, accurate wall damage assessment and repair guidance for Sri Lankan buildings. From data collection to deployment preparation, each step was designed to ensure robustness, scalability, and usability. The chosen technologies and libraries balance performance

and maintainability, with planned fine-tuning and deployment to achieve  $\text{mAP} > 0.8$  and real-time inference.

Table 1: Tools and Technologies Used

| Tool/Technology                  | Type      | Role in Project                                                                               |
|----------------------------------|-----------|-----------------------------------------------------------------------------------------------|
| Python                           | Language  | Core programming for model development, preprocessing, cost estimation, and API interactions. |
| Ultralytics YOLO (8.3.133)       | Library   | Object detection with YOLOv8 nano for damage localization.                                    |
| TensorFlow/Keras                 | Library   | ResNet-50 model training and classification of damage types.                                  |
| NumPy                            | Library   | Numerical computations (e.g., area calculations, confidence scores).                          |
| OpenCV                           | Library   | Image preprocessing (resizing, augmentation).                                                 |
| Pandas                           | Library   | Metadata and result handling (CSVs).                                                          |
| Scikit-learn                     | Library   | Class weight balancing for ResNet-50.                                                         |
| LabelImg                         | Tool      | Bounding box annotation for YOLOv8 dataset.                                                   |
| Conda                            | Tool      | Environment management (' <i>wall<sub>damage</sub></i> ', ' <i>LLM<sub>env</sub></i> ').      |
| Jupyter Notebooks                | Tool      | Exploratory data analysis and prototyping.                                                    |
| Node.js with NestJS              | Framework | Backend API for image uploads and script execution.                                           |
| python-shell                     | Library   | Executes Python scripts from Node.js back-end.                                                |
| @nestjs/platform-express, Multer | Library   | File upload handling in NestJS.                                                               |
| google.generativeai              | Library   | Interfaces with Gemini API for explanations.                                                  |
| Windows OS                       | Platform  | Development and training environment (CPU-based).                                             |
| Git                              | Tool      | Version control for scripts and models (planned).                                             |
| ONNX                             | Format    | Model export for cross-platform deployment (planned).                                         |
| FastAPI                          | Framework | Scalable API for serving predictions (planned).                                               |
| React.js                         | Framework | Front-end development for user interface (planned).                                           |
| MongoDB                          | Database  | Storage for inference results and user data (planned).                                        |
| Albumentations                   | Library   | Advanced image augmentations for fine-tuning (planned).                                       |
| Weights & Biases                 | Tool      | Experiment tracking and visualization (planned).                                              |
| Prometheus + Grafana             | Tool      | Monitoring production performance (planned).                                                  |
| AWS/GCP                          | Platform  | Cloud hosting and inference scaling (planned).                                                |
| Docker                           | Tool      | Containerization for consistent deployment (planned).                                         |